

Universidad San Mateo

Gestión de Bases de Datos (MySQL)

Guía Proyecto Integrador

**(Diagnóstico de rendimiento + Optimización + Seguridad + Backup/Restore
+ Hash de contraseñas)**



Nota: los primeros tres entregables no los tuve en cuenta pues ustedes ya los tienen listos 😊

Objetivo de la Guía

- 1) diagnosticar consultas con EXPLAIN
- 2) proponer y aplicar mejoras (índices)
- 3) configurar seguridad básica (usuarios, permisos y vistas)
- 4) ejecutar un proceso de respaldo y recuperación (backup/restore),
- 5) proteger contraseñas en un escenario hipotético mediante SALT + HASH.

Entregables

1. Informe de rendimiento
2. Documento backup/restore
3. Para rendimiento: mínimo 5 consultas con EXPLAIN ANTES y DESPUÉS + cambio aplicado + conclusión.
4. Evidencia: captura de resultados (EXPLAIN, tiempos, SHOW GRANTS, validación post-restore).
5. Trabajar sobre la base: san_mateo_academico posteriormente aplicar estos principios a la base de datos de su **proyecto Integrador**

Preparación:

0.1 Selecciona la base de datos:

```
USE san_mateo_academico;
```

0.2 Evidencia de versión (ambiente):

```
SELECT VERSION();
```

0.3 Conteos (tamaño lógico del dataset):

```
SELECT COUNT(*) AS students FROM students;
SELECT COUNT(*) AS enrollments FROM enrollments;
SELECT COUNT(*) AS assessments FROM assessments;
SELECT COUNT(*) AS grades FROM grades;
```

Evidencia obligatoria: captura de los conteos y la versión.

Diagnóstico de rendimiento con EXPLAIN (5 consultas)

EXPLAIN muestra el plan de ejecución: cómo MySQL va a buscar los datos. Para cada consulta debes guardar: type, key, rows, Extra (ANTES y DESPUÉS).

Consulta 1: Matrículas por estudiante

```
EXPLAIN
SELECT e.enrollment_id, e.status, e.enrolled_at
FROM enrollments e
WHERE e.student_id = 1;
```

Cambio (índice):

```
CREATE INDEX idx_enrollments_student ON enrollments(student_id);
```

EXPLAIN (DESPUÉS):

```
EXPLAIN
SELECT e.enrollment_id, e.status, e.enrolled_at
FROM enrollments e
WHERE e.student_id = 1;
```

Consulta 2: Lista de clase por sección (JOIN simple)

```
EXPLAIN
SELECT e.enrollment_id, s.doc_number, s.first_name, s.last_name
FROM enrollments e
JOIN students s ON s.student_id = e.student_id
WHERE e.section_id = 1;
```

Cambio (índice):

```
CREATE INDEX idx_enrollments_section ON enrollments(section_id);
```

EXPLAIN (DESPUÉS):

```
EXPLAIN
```

```
SELECT e.enrollment_id, s.doc_number, s.first_name, s.last_name
FROM enrollments e
JOIN students s ON s.student_id = e.student_id
WHERE e.section_id = 1;
```

Consulta 3: Calendario de evaluaciones por periodo (JOIN + filtro)

```
EXPLAIN
```

```
SELECT ap.code, c.code, cs.group_code, a.name, a.due_date
FROM assessments a
JOIN course_sections cs ON cs.section_id = a.section_id
JOIN courses c ON c.course_id = cs.course_id
JOIN academic_periods ap ON ap.period_id = cs.period_id
WHERE ap.code = '2026-1'
      AND a.due_date IS NOT NULL
ORDER BY a.due_date;
```

Cambios (índices):

```
CREATE INDEX idx_assessments_section ON assessments(section_id);
CREATE INDEX idx_sections_period ON course_sections(period_id);
```

EXPLAIN (DESPUÉS):

```
EXPLAIN
```

```
SELECT ap.code, c.code, cs.group_code, a.name, a.due_date
FROM assessments a
JOIN course_sections cs ON cs.section_id = a.section_id
JOIN courses c ON c.course_id = cs.course_id
JOIN academic_periods ap ON ap.period_id = cs.period_id
WHERE ap.code = '2026-1'
      AND a.due_date IS NOT NULL
ORDER BY a.due_date;
```

Consulta 4: Notas por matrícula

```
EXPLAIN
```

```
SELECT g.grade_id, g.score, a.name, a.weight_pct
FROM grades g
JOIN assessments a ON a.assessment_id = g.assessment_id
WHERE g.enrollment_id = 1;
```

Cambio (índice):

```
CREATE INDEX idx_grades_enrollment ON grades(enrollment_id);
```

EXPLAIN (DESPUÉS):

```
EXPLAIN
SELECT g.grade_id, g.score, a.name, a.weight_pct
FROM grades g
JOIN assessments a ON a.assessment_id = g.assessment_id
WHERE g.enrollment_id = 1;
```

Consulta 5: Conteo de inscritos por sección (GROUP BY)

```
EXPLAIN
SELECT section_id, COUNT(*) AS total
FROM enrollments
WHERE status = 'ENROLLED'
GROUP BY section_id;
```

Cambio (índice compuesto):

```
CREATE INDEX idx_enrollments_status_section ON enrollments(status,
section_id);
```

EXPLAIN (DESPUÉS):

```
EXPLAIN
SELECT section_id, COUNT(*) AS total
FROM enrollments
WHERE status = 'ENROLLED'
GROUP BY section_id;
```

Producto de esta parte: tabla comparativa antes/después (consulta, key/type/rows/extra, índice aplicado, conclusión).

Seguridad (usuarios, permisos y vistas)

2.1 Crear usuario de solo lectura para reportes:

```
CREATE USER 'report_user'@'%' IDENTIFIED BY 'StrongPass_2026!';
GRANT SELECT ON san_mateo_academico.* TO 'report_user'@'%';
FLUSH PRIVILEGES;
```

2.2 Crear una vista pública sin email (protege columnas sensibles):

```
CREATE OR REPLACE VIEW v_students_public AS
SELECT student_id, doc_type, doc_number,
       CONCAT(first_name, ' ', last_name) AS full_name,
       status
FROM students;
```

2.3 Evidencia de permisos asignados:

```
SHOW GRANTS FOR 'report_user'@'%';
```

Backup y Restore (respaldo y recuperación)

Exportar BD completa a un archivo .sql (Self-Contained File)

Abre MySQL Workbench y conéctate a tu servidor.

En el menú superior: Server → Data Export

En Schemas, marca la base: san_mateo_academico

En Objects to Export:

- ☒ Check Dump Structure and Data (estructura + datos)
- ☒ Check Dump Stored Procedures and Functions
- ☒ Check Dump Triggers
- ☒ Check Dump Events

En Export Options:

Selecciona Export to Self-Contained File

Elige ruta y nombre, por ejemplo:

C:\backups\san_mateo_academico_backup.sql

Click en Start Export

Evidencia para el proyecto

Captura de pantalla de la ventana “Data Export” con la BD seleccionada y opciones marcadas.

Captura del mensaje “Export completed” o del log del export.

Verificación rápida del backup (desde Workbench)

Como el backup es un archivo .sql, la verificación básica es:

Revisa en el explorador de archivos y confirma que el .sql existe y tiene tamaño razonable (no 0 KB).

Restore (Recuperación) desde MySQL Workbench

Menú: Server → Data Import

Selecciona: Import from Self-Contained File

elige tu archivo: san_mateo_academico_backup.sql

En Default Target Schema

Click en Start Import

Evidencia para el proyecto

Captura de la ventana “Data Import” con el archivo seleccionado

Captura del mensaje “Import completed” o del log

Validación post-restore

Después del import, ejecutan estas sentencias:

```
USE san_mateo_academico; -- o san_mateo_restore_test
```

```
SHOW TABLES;
```

```
SELECT COUNT(*) AS students FROM students;
```

```
SELECT COUNT(*) AS enrollments FROM enrollments;
```

```
SELECT COUNT(*) AS assessments FROM assessments;
```

```
SELECT COUNT(*) AS grades FROM grades;
```

Qué deben evidenciar:

Que las tablas existen

Que los conteos son razonables (no cero si antes había datos)

Simulación de “desastre”

evidencia del proyecto

Backup hecho (Data Export)

Ejecutar en Query Tab:

```
DROP DATABASE san_mateo_academico;
```

Restore con Server → Data Import

Validación con SHOW TABLES + COUNT(*)

Capturas de cada paso.

Protección de contraseñas (caso hipotético user/pass)

Regla: no almacenar contraseñas en texto plano. En este laboratorio se aplica un enfoque didáctico: SALT + SHA2 (hash no reversible).

4.1 Crear tabla de usuarios de aplicación:

```
CREATE TABLE IF NOT EXISTS app_users (  
  user_id INT AUTO_INCREMENT PRIMARY KEY,  
  username VARCHAR(60) NOT NULL UNIQUE,  
  password_salt VARBINARY(16) NOT NULL,  
  password_hash VARBINARY(32) NOT NULL,  
  created_at DATETIME NOT NULL DEFAULT CURRENT_TIMESTAMP  
);
```

4.2 Procedimiento para crear usuario (genera salt + hash):

```
DROP PROCEDURE IF EXISTS sp_create_user;
```

```
DELIMITER $$
```

```
CREATE PROCEDURE sp_create_user(  
  IN p_username VARCHAR(60),  
  IN p_password VARCHAR(255)  
)  
BEGIN  
  DECLARE v_salt VARBINARY(16);  
  DECLARE v_hash VARBINARY(32);  
  
  SET v_salt = RANDOM_BYTES(16);  
  SET v_hash = UNHEX(SHA2(CONCAT(HEX(v_salt), p_password), 256));
```

```

    INSERT INTO app_users(username, password_salt, password_hash)
    VALUES (p_username, v_salt, v_hash);
END $$

```

```

DELIMITER ;

```

4.3 Función para validar contraseña (recalcula hash y compara):

```

DROP FUNCTION IF EXISTS fn_check_password;

```

```

DELIMITER $$

```

```

CREATE FUNCTION fn_check_password(p_username VARCHAR(60), p_password
VARCHAR(255))
RETURNS TINYINT(1)
DETERMINISTIC
READS SQL DATA
BEGIN
    DECLARE v_salt VARBINARY(16);
    DECLARE v_hash VARBINARY(32);
    DECLARE v_calc VARBINARY(32);

    SELECT password_salt, password_hash
        INTO v_salt, v_hash
    FROM app_users
    WHERE username = p_username
    LIMIT 1;

    IF v_salt IS NULL THEN
        RETURN 0;
    END IF;

    SET v_calc = UNHEX(SHA2(CONCAT(HEX(v_salt), p_password), 256));

    IF v_calc = v_hash THEN
        RETURN 1;
    END IF;

    RETURN 0;
END $$

```

```

DELIMITER ;

```

4.4 Pruebas:


```
CALL sp_create_user('ana', 'MiClave123!');  
SELECT fn_check_password('ana', 'MiClave123!') AS ok;  
SELECT fn_check_password('ana', 'otra') AS ok;
```

Evidencias que se deben anexar al entregable para la sustentación del proyecto

6. Captura de SELECT VERSION() y conteos (tamaño del dataset).
7. Capturas de EXPLAIN antes y después para las 5 consultas.
8. Captura del CREATE INDEX aplicado (o el script).
9. Captura de SHOW GRANTS del usuario report_user.
10. Captura/archivo del backup y evidencia de restore + SHOW TABLES + COUNT(*) post-restore.
11. Captura de pruebas del hashing (fn_check_password retornando 1 y 0).

EXITOS 😊